

DeductIT: An Interactive Natural Deduction Game

Software Verification Final Year Project

Rory O'Donoghue

21420506

Final year project 2025/2026

Web Development Project

15 ECTS

B.Sc. Computer Science and Software Engineering



**Department of Computer Science,
Maynooth University,
Maynooth,
Co. Kildare,
Ireland**

Gitlab Link: [RORY O DONOGHUE / FinalProject · GitLab](#)

**A thesis submitted in partial fulfilment of the requirements for the B.Sc. Computer Science
And Software Engineering.**

Supervisor: Dr. Rosemary Monahan

Abstract

This project presents **DeductIT**, an interactive browser-based educational game designed to support the learning of propositional natural deduction. Natural deduction is a formal proof system widely used in logic and computer science education, but many students find traditional proof exercises difficult to engage with. DeductIT addresses this challenge by introducing a visual and interactive environment where users construct proofs step-by-step using logical rules.

The system is implemented using a React-based frontend and a JavaScript rule-validation engine that checks the correctness of each proof step. Logical expressions are built through a structured interface to prevent malformed input, and exercises are organised into progressive difficulty tiers derived from natural deduction problem sets.

An evaluation study was conducted with users to assess usability, learning support, and engagement. Results indicate that the system improves accessibility and provides a more interactive learning experience compared to traditional paper-based exercises.

The project demonstrates how interactive web technologies can support the teaching of formal logic and outlines potential future work including user accounts, backend storage, and AI-generated exercises.

Contents

Chapter 1: Introduction	3
Summary	3
1.1 Overview	3
1.2 Motivation	3
1.3 Approach	4
1.4 Metrics	4
1.5 Project	4
Chapter 2: Technical Background	4
Summary	4
2.1 Topic Material	4
2.2 Technical Material	6
Chapter 3: The Problem	7
Summary	7
3.1 Project UML Documentation	7
3.2 Problem Analysis	7
Chapter 4: The Solution	8
Summary	8
4.1 Analytical Work	8
4.2 Architectural Level	9
4.3 High-Level Design	9
4.4 Low-Level Design	9
4.5 Implementation	11
Chapter 5: System Evaluation	11

Summary.....	11
5.1 System Verification.....	12
5.2 User Evaluation Methodology	12
5.3 Evaluation Results	12
5.4 Evaluation Summary.....	15
Chapter 6: Conclusions	15
6.1 Overview	15
6.2 Project Contributions	15
6.4 Limitations	15
6.5 Future Work	16
6.6 Reflection on Project Goals.....	16
Final Statement	17
References	17
Appendices	18

Chapter 1: Introduction

Summary

This chapter introduces the domain of natural deduction and outlines the motivation for developing an interactive, browser-based educational game to support its learning. It presents the technical problem addressed by the project, summarises the approach taken to design and implement the system, defines the metrics used to evaluate success, and identifies the key contributions of the work. Finally, it describes the structure of the thesis.

1.1 Overview

Natural deduction is a formal proof system used extensively in computer science, mathematics, and logic education. It provides a rule-based framework for deriving conclusions from premises using inference rules such as conjunction introduction, implication elimination, and negation introduction. Within undergraduate computer science curricula, natural deduction plays a central role in subjects such as discrete mathematics, formal methods, and theoretical computer science.

Despite its importance, natural deduction is widely regarded as conceptually challenging. Students must manage nested assumptions, maintain awareness of scope, correctly apply inference rules, and construct well-formed logical expressions. Traditional teaching approaches often rely on static textbook examples or text-based proof editors, which can be difficult for beginners to navigate.

This project aims to design and implement a web-based interactive system that improves engagement and conceptual clarity in natural deduction learning. The system combines formal logic foundations with modern web technologies and game design principles to create an educational environment that is both rigorous and accessible.

1.2 Motivation

Learning natural deduction requires more than memorising inference rules; it also requires an understanding of proof structure, strategic reasoning, and syntactic accuracy. Students often struggle not because individual rules are difficult, but because managing the overall structure of a proof becomes cognitively demanding.

A review of existing tools such as Logiko and other browser-based proof editors revealed several limitations. Many interfaces are text-heavy and minimally interactive, feedback typically focuses only

on correctness rather than explanation, and subproof scope is usually represented only through indentation. In addition, exercises are generally static and identical for all users, while features such as progression systems or performance tracking are limited.

As a result, many existing systems function primarily as correctness checkers rather than interactive learning environments. They verify whether a proof step is logically valid but provide limited support for engagement or conceptual understanding.

This project therefore addresses the gap between formal logic proof tools and engaging educational platforms by combining modern web technologies with formally grounded logical rules to create a more effective learning environment.

1.3 Approach

The project follows a structured software engineering approach including analysis of existing natural deduction tools, identification of system requirements, design of a modular web-based architecture, implementation of a rule-based validation engine, and user evaluation.

The system is implemented as a browser-based application using React. Logical expressions are constructed through a button-based builder to prevent malformed input, while a custom JavaScript checker validates rule applications incrementally. Exercises are drawn from structured natural deduction problem sets, and rule behaviour is informed by the formally verified NaDeA framework. The design prioritises correctness, usability, visual clarity, and extensibility.

1.4 Metrics

The success of the system will be evaluated according to several measurable criteria:

- **Correctness Accuracy:** The system must correctly validate legal inference steps and reject invalid ones.
- **Syntax Enforcement:** The expression builder must prevent malformed logical formulas.
- **User Task Completion Rate:** Percentage of users successfully completing assigned proof exercises.
- **Comparative Usability Feedback:** User-reported clarity and engagement compared with traditional tools.
- **System Responsiveness:** Performance under typical interaction conditions.

These metrics will be analysed in Chapter 5 following user-based evaluation.

1.5 Project

The key achievements of this project include:

- Design and implementation of an interactive natural deduction game.
- Development of a strict syntactic expression builder preventing malformed formulas.
- Implementation of a modular natural deduction rule-checking engine.
- Integration of formally grounded logic rules inspired by the NaDeA Isabelle framework.

Together, these components form a scalable educational platform bridging formal logic and modern web interaction design.

Chapter 2: Technical Background

Summary

This chapter outlines the theoretical and technical background for the natural deduction game. It reviews the foundations of natural deduction, examines existing proof tools and their limitations, and introduces key technical concepts such as web-based architecture and logical formula representation that inform the system design.

2.1 Topic Material

2.1.1 Natural Deduction as a Formal Proof System

Natural deduction is a formal proof system introduced by Gerhard Gentzen [1] in the 1930s to represent logical reasoning in a structured and rule-based manner. Unlike semantic approaches such as

truth tables or model checking, natural deduction operates syntactically: conclusions are derived through the application of inference rules to previously established statements.

Each logical connective has associated introduction and elimination rules. For example:

- **Conjunction Introduction ($\wedge I$)** allows the derivation of $P \wedge Q$ from P and Q .
- **Implication Elimination ($\rightarrow E$)**, also known as Modus Ponens, derives Q from $P \rightarrow Q$ and P .
- **Implication Introduction ($\rightarrow I$)** discharges an assumption after deriving a consequent, enabling the conclusion $P \rightarrow Q$.

A full description of all rules can be found in supporting document “user_documentation”.

A defining feature of natural deduction is assumption discharge. Temporary assumptions may be introduced to derive implications, after which they are removed. This creates nested proof structures and scope hierarchies that must be carefully maintained.

Although conceptually elegant, natural deduction presents learning challenges including: Tracking nested assumptions, managing line dependencies, recognising valid rule application contexts, maintaining syntactic correctness and planning overall proof strategy.

Students often struggle not because individual rules are complex, but because the structural organisation of proofs becomes cognitively demanding.

2.1.2 Review of Existing Natural Deduction Systems

A survey of existing natural deduction tools was conducted to identify current approaches and limitations. Systems reviewed include Logiko [2], Natural Deduction Proof Editor [3], Pandora [4] and QED [5].

Several common limitations were identified. First, visual engagement is often low, as many interfaces rely heavily on static text-based layouts that provide little visual guidance during proof construction.

Examples of these interfaces can be seen in the appendix figures.

Second, pedagogical feedback is limited. Many systems simply indicate whether a step is valid or invalid without explaining the reasoning behind the error. This can make it difficult for beginners to understand mistakes or progress in their learning. An example of this behaviour is shown in Figure A.1 (Appendix A).

Another limitation is the lack of adaptive difficulty. Most systems present the same set of exercises to all users regardless of experience level. As illustrated in Figure A.2 (Appendix A), the exercises remain static and do not adjust based on user progress.

Finally, performance tracking is often minimal. Systems such as QED operate largely as stateless proof checkers and do not provide mechanisms for monitoring user improvement or identifying recurring weaknesses.

These observations highlight an opportunity to combine formal proof validation with more interactive design and structured progression mechanisms. Further analysis of these systems is provided in the supporting documents “Analysis_of_other_similar_systems”, “Deeper analysis of logiko” and “what I can build on”

2.1.3 Exploration of Formal Theorem Provers

During the system design phase, formal theorem provers were investigated as potential mechanisms for ensuring machine-verified correctness of user-constructed proofs.

The systems explored included Isabelle/HOL, Lean, Coq, Lightweight embedded proof checkers

Isabelle/HOL

Isabelle/HOL [6] is a mature interactive theorem prover widely used in formal methods research and formalised mathematics. It provides a higher-order logic framework for constructing machine-verified proofs and supports structured proof development through a combination of automated and interactive reasoning tools.

The NaDeA [7](Natural Deduction Assistant) framework, implemented in Isabelle, demonstrates formally verified natural deduction systems for propositional and predicate logic. NaDeA illustrates how theorem provers can support the teaching of formal reasoning by providing machine-checked validation of proof steps.

A potential integration strategy was considered in which user-constructed proofs would be translated into Isabelle syntax and verified via a backend service.

A full exploration of Isabelle as a backend logic checker can be found in supporting document “Isabelle_implementation”.

Exploration of Lean [8]

Lean, developed by Microsoft Research, is a modern interactive theorem prover based on dependent type theory. It has gained significant adoption within the formal mathematics community and is supported by a growing ecosystem of tools, libraries, and educational resources.

Lean provides a tactic-based interaction model that allows users to construct proofs incrementally while receiving automated assistance from the proof engine. Its integration with development environments such as Visual Studio Code has also made it accessible for teaching formal reasoning and program verification.

These systems demonstrate how machine-verified proof environments can support the construction and validation of formal logical arguments.

A full exploration of Lean as a backend logic checker can be found in supporting document “Lean_implementation”.

2.2 Technical Material

2.2.1 Web Application Architecture

Web-based educational systems offer advantages including cross-device accessibility, ease of deployment, and elimination of installation requirements. Modern web frameworks support component-based design, enabling modular separation of interface elements and interactive behaviour. Client-side state management allows dynamic updates in response to user interaction, making it well suited to incremental proof construction environments. Web technologies therefore provide a natural platform for interactive logic education tools.

2.2.2 Logical Formula Representation

To validate natural deduction steps programmatically, formulas must be stored in structured form. Many systems use parsers that convert text input into Abstract Syntax Trees (ASTs).

To ensure that logical expressions can be reliably processed, formulas must follow strict syntactic rules. The expression builder therefore enforces several constraints during formula construction. It guarantees balanced parentheses, valid connective ordering, and restricted atomic symbol usage. For example, balanced parentheses ensure that valid expressions are accepted, while malformed expressions are rejected. Valid connective ordering ensures that connectives appear between well-formed formulas. Finally, atomic symbols are restricted to predefined propositional variables preventing unsupported symbols or malformed inputs from being entered. Examples of both valid and invalid representation can be seen later in the paper.

Internally, formulas are stored as structured JavaScript objects representing logical operators and operands. This allows structural comparison rather than simple string matching.

This strict syntactic enforcement strategy was selected to prevent syntactic errors from propagating into later rule validation stages, simplify rule-checking logic and prevents unwanted characters being entered into the system

Chapter 3: The Problem

Summary

This chapter defines the technical problem addressed by the project and outlines the system requirements. It identifies the primary system actors and presents functional, non-functional, and UML-based models to provide a structured analysis of the problem domain.

3.1 Project UML Documentation

The system is an interactive web-based educational tool designed to allow users to construct and validate natural deduction proofs. From a requirements perspective, the system involves one primary actor and several core system interactions. The UML artefacts supporting this system analysis are provided in Appendix A.

3.1.1 System Actors

The primary actor is: User (Student) – interacts with the system to construct and validate proofs, complete exercises, and progress through levels while tracking their score.

Future extensions could introduce additional actors (e.g., Administrator or Instructor), but the implemented scope focuses solely on student interaction.

3.1.2 Use Case Overview

These interactions are illustrated in the UML Use Case Diagram (see Figure B.1 in Appendix B). The major use cases identified are: Load a proof exercise, view premises and target conclusion, construct logical expressions, select inference rule, submit proof line, receive validation feedback, complete proof and view progression to next level.

3.1.3 Primary Use Case: Construct and Validate Proof Line

Preconditions: A proof exercise is loaded and premises and target conclusion are visible.

Main Flow: User selects logical symbols using the expression builder->User selects an inference rule->User references previous lines (if required).->User submits the line.->System validates the rule application.->System displays feedback.->If valid, line is added to proof state.

Postconditions: Proof state is updated and validation feedback is stored/displayed.

3.1.4 Activity Flow

The activity sequence is illustrated in the Activity Diagram (see Figure B.2 in Appendix B)

The high-level activity sequence is: Display exercise->User constructs formula->User selects rule->System validates rule->If valid → add line or If invalid → display error->Repeat until target conclusion reached

3.1.5 Conceptual Class Model

The relationships between these entities are illustrated in the UML Class Diagram (see Figure B.3 in Appendix B).

From a problem perspective (independent of implementation), the system consists of the following conceptual entities:

• Exercise

- Contains premises
- Contains a target conclusion
- Has an associated difficulty level

• ProofLine

- Contains a logical formula
- Has a rule label
- References prior lines
- Has a scope level

• Proof

- Represents an attempt to solve an Exercise
- Contains multiple ProofLines
- Is associated with a single Exercise

3.2 Problem Analysis

3.2.1 Functional Requirements

The system must: Allow users to load predefined natural deduction exercises, display premises (where necessary) and a target conclusion, allow users to construct logical formulas, allow selection of inference rules for each proof line, allow referencing of previously derived lines, validate rule applications against propositional natural deduction rules, provide feedback indicating whether a proof step is valid or invalid, allow progression between exercises upon successful completion.

3.2.2 Non-Functional Requirements

- **Correctness:** The system must ensure logically valid rule applications.
- **Usability:** The system must minimise syntactic errors and reduce cognitive overload.
- **Responsiveness:** Feedback must be delivered without noticeable delay.
- **Accessibility:** The system must operate in modern web browsers without installation.
- **Extensibility:** The system should allow additional inference rules or exercise types to be incorporated.

3.2.3 Constraints

Constraints define limitations imposed on the problem space. The system must support full propositional natural deduction. Predicate logic support is limited to structural validation. Exercises are predefined rather than dynamically generated. The system must prevent malformed logical expressions. The system must operate within a web-based environment.

3.2.4 Problem Complexity

Natural deduction introduces significant structural complexity due to factors such as nested assumptions and scope rules, the existence of multiple valid proof paths, rule-specific structural requirements, and dependency tracking between proof lines. From a system perspective, supporting these features requires mechanisms for structured formula representation, validation of line references, scope tracking, and prevention of illegal inference patterns. The core technical problem can therefore be summarised as designing a system that enables users to construct natural deduction proofs while ensuring rule correctness, structural consistency, and syntactic validity within an interactive educational environment. The analysis presented in this chapter highlights the key technical challenges involved in supporting natural deduction in such a system. Addressing these challenges requires a design capable of representing logical formulas, validating inference rules, and maintaining proof state during user interaction. The following chapter therefore presents the design and implementation of the DeductIT system.

Chapter 4: The Solution

Summary

This chapter describes the design and implementation of the interactive natural deduction game. It outlines the system architecture, component design, and the algorithmic logic used for proof validation, while discussing key implementation decisions and trade-offs.

4.1 Analytical Work

The central challenge of this project is representing and validating natural deduction proofs computationally. Each proof step must satisfy several conditions: the formula must be syntactically valid, the selected inference rule must be applied correctly, referenced proof lines must exist, and scope constraints must be respected when assumptions are introduced or discharged.

To support this, each proof line is stored with a structured representation of the formula, the applied rule, references to previous lines, and the current scope level. When a user submits a new proof step, the system checks that referenced lines appear earlier in the proof, verifies that the rule-specific structural conditions are satisfied, and ensures that scope boundaries are not violated.

For example, implication elimination ($\rightarrow E$) requires that one referenced line has the form $P \rightarrow Q$, that another referenced line matches P , and that the derived conclusion is Q . This validation strategy forms the basis of the rule-checking engine implemented in the system.

4.2 Architectural Level

The architectural block diagram is provided in Appendix A (Figure B.4).

The system follows a single-page application (SPA) architecture with a React-based client interface [9] and a lightweight Node/Express backend responsible for structured proof validation.

The major architectural components are:

- **User Interface Layer** (game board, proof panel, controls)
- **Expression Builder** (button-based formula construction)
- **Proof State Manager** (tracks proof lines, indentation, current expression)
- **Rule Validation Engine** (validates submitted lines)
- **Level Manager** (loads exercises and manages progression)
- **Tutorial Overlay** (first-time guidance persisted via localStorage)
- **Score Tracker** (updates and displays progress)

The architecture follows a unidirectional data flow: User Input → State Update → Validation → Render Update

The React frontend manages interactive state transitions, while the backend validation service ensures rule-specific structural checking. This separation preserves responsiveness while maintaining correctness.

4.3 High-Level Design

The primary gameplay interface, including premises, target conclusion, and proof workspace, is illustrated in Figure E.1 (Appendix E).

At a high level, the application is composed of modular React components:

- **App** – root controller handling routing and layout
- **GameBoard** – main game container displaying premises, goal, and proof workspace
- **TutorialOverlay** – onboarding modal

The tutorial overlay presented to new users is shown in the images in Figure E.6 (Appendix E).

- **Proof Display** – renders proof lines with indentation to represent subproof scope
- **Expression Builder** – structured construction of logical formulas
- **Rule Selector** – allows inference rule selection
- **Level Progression Module** – advances through predefined exercises

Successful proof completion and level progression are illustrated in Figure E.5 (Appendix E).

The logical data model consists of:

- | | | |
|-----------------------|----------------|--------------------|
| • Exercise | • Proof | • ProofLine |
| ○ Premises | ○ Array of | ○ Formula |
| ○ Target conclusion | ProofLine | (structured |
| ○ Difficulty metadata | objects | object) |

All state transitions occur through controlled React updates, ensuring predictable rendering behaviour and consistency across components.

4.4 Low-Level Design

4.4.1 Formula Representation

Logical formulas are represented as structured JavaScript objects rather than raw strings. An example of this is shown in figure C.1 (appendix C)

This structured representation enables: Structural comparison, recursive validation and elimination of syntactic ambiguity. By avoiding string-based comparison, the system ensures that logical equivalence is determined structurally rather than textually.

4.4.2 Expression Builder Logic

The expression builder enforces syntactic correctness through controlled input. It guarantees:

1. Balanced parentheses: $(P \wedge Q) \rightarrow R$ VALID

$(P \wedge Q \rightarrow R$ INVALID

2. Valid connective placement, ensuring that logical connectives (e.g., $\wedge$, $\vee$, $\rightarrow$) appear only in valid positions."

$P \wedge Q$ VALID

$\wedge P Q$ INVALID

3. Restricted atomic symbol usage

$P \wedge Q$ VALID

$P \wedge X1\$$ INVALID

Each button press updates a structured formula state rather than appending text. This prevents malformed expressions from entering the validation pipeline.

4.4.3 Rule Validation Algorithm

Each inference rule is implemented as a dedicated validation function.

Example pseudocode for implication elimination is shown in figure C.2(appendix C.2)

All rules follow a similar structural validation pattern.

The engine additionally verifies:

- Referenced lines exist
- No future line dependencies
- The final derived formula matches the goal

4.4.4 Scope Management

Scope is tracked using a numeric depth counter. Each ProofLine contains a scopeLevel value.

Validation partially enforces scope constraints by:

- Preventing direct reference to inaccessible lines
- Tracking nested subproof levels

The visual representation of nested assumptions and indentation-based scope tracking and the buttons used to do so are illustrated in Figures E.2 and E.3 (Appendix E).

Full discharge enforcement for all scope-sensitive rules is identified as future work. However, current structural enforcement provides sufficient correctness guarantees for supported propositional exercises.

4.4.5 Rule and Level design

4.4.5.1 Inference Rule Specification

The inference rules implemented in the system follow standard formulations of propositional natural deduction commonly used in introductory logic education. These rules allow new statements to be derived from previously established formulas through structured transformations, including conjunction introduction ($\wedge I$), conjunction elimination ($\wedge E$), implication elimination ($\rightarrow E$), and implication introduction ($\rightarrow I$).

The rule definitions used in this project are consistent with those described in the Open Logic Project natural deduction framework, which provides a formal specification of rule behaviour and proof construction.

Each rule is implemented as a validation function within the rule-checking engine. These functions examine the structure of referenced proof lines and verify that the derived formula satisfies the structural requirements of the selected rule. For example, implication elimination ($\rightarrow E$) checks that one referenced line has the form $P \rightarrow Q$, another matches P , and that the resulting formula is Q .

This mapping of logical rules to structured validation functions ensures that proof steps follow the formal behaviour of natural deduction while remaining computationally efficient for interactive use. A further description of all inference rules can be found in "Natural Deduction," by Carnegie Mellon University. [10]

4.4.5.2 Exercise and Level Design

The exercise set was designed to reflect the progression typically used in introductory natural deduction teaching materials. Early levels focus on simple single-step derivations, allowing users to become familiar with the interface and rule selection process. Later exercises introduce longer proof chains and nested assumptions that require combining multiple inference rules.

Each exercise is defined by a structured goal consisting of premises and a target conclusion. Difficulty is approximated using an estimated step count stored in the level definition. This progression-based structure allows users to gradually develop familiarity with the rule system while maintaining a clear sense of advancement through the game. The original question base was inspired by “The Natural Deduction Pack” which can be found in supporting documents. It was later deemed that the level of difficulty of these questions was out of scope for this project.

4.5 Implementation

4.5.1 React State Management

The system uses React’s useState hooks to maintain: Current proof state, current formula under construction, selected inference rule, current level index and tutorial visibility state. State updates trigger automatic re-rendering of affected components, ensuring consistency across the interface.

4.5.2 JSON Level Structure

Exercises are stored in a JSON file loaded at runtime (Natural_Deduction_levels.json). Each entry includes a single goal string in the form “premise1, premise2 $\vdash$ conclusion (n)”, where n is an optional step-count annotation used for difficulty feedback.

An image of the structure is shown in figure (D.1 Appendix D)

This structure enables: Easy addition of new exercises, separation of content from validation logic and lightweight deployment without database dependency

4.5.3 Error Handling

If validation fails:

- The proof state remains unchanged
- An explanatory message is displayed
- The user may attempt correction

This approach prevents invalid proof states and maintains structural consistency.

4.5.4 Trade-offs

Several architectural trade-offs were made:

1. No External Theorem Prover Integration

Although integration with systems such as Isabelle or Lean would provide machine-verified guarantees, this would significantly increase architectural complexity, require persistent backend infrastructure, and reduce responsiveness. A structured local rule-validation engine was therefore adopted. Further details on why neither Isabelle or Lean were used can be found in supporting documents “lean_implementation_issues” and “Isabelle_Implementation_Issues”.

2. Structured Input Instead of Full Parser

A button-based expression builder was used instead of free-text parsing. While this limits expressiveness, it prevents malformed expressions and reduces validation complexity.

3. Lightweight Backend Service

A minimal Node/Express backend was used exclusively for structured validation. This balances correctness enforcement with deployment simplicity.

An example of rule-specific validation feedback is shown in Figure E.4 (Appendix D).

Chapter 5: System Evaluation

Summary

This chapter evaluates the implemented system through both technical verification and user-based evaluation. The objective is to determine whether the system functions correctly, whether the interface

supports user interaction effectively, and whether the system improves understanding of natural deduction concepts.

Evaluation was conducted using a structured questionnaire completed by ten participants following interaction with the system.

5.1 System Verification

Before conducting user evaluation, the system was verified to ensure that the core functionality operated as intended. The following aspects were tested:

5.1.1 Rule Validation

The rule validation engine correctly verifies inference rules by checking the structural relationships between formulas and referenced proof lines. For example, implication elimination ($\rightarrow E$) requires that one referenced line has the form $P \rightarrow Q$ and that another referenced line matches P , producing Q as the conclusion.

5.1.2 Syntax Enforcement

The expression builder prevents malformed formulas by restricting input through predefined buttons. This ensures balanced parentheses, valid connective placement, and correct atomic symbol usage.

5.1.3 Proof State Management

The proof workspace correctly maintains proof state across multiple lines, ensuring that references only apply to previously derived statements.

5.1.4 Feedback System

The system provides immediate feedback when a rule is applied incorrectly. For example, when conjunction introduction is used incorrectly, the system displays an explanatory error message indicating that the conclusion must match the conjunction of the referenced lines.

Screenshots demonstrating rule validation feedback and proof progression are shown in figure E.4 in Appendix E.

5.2 User Evaluation Methodology

User evaluation was conducted with ten participants who were asked to interact with the DeductIT system and complete a short evaluation sheet. Participants were provided with a set of natural deduction exercises and asked to complete at least one proof using the interface. Following interaction with the system, participants completed an evaluation questionnaire consisting of:

- Six Likert-scale questions measuring usability and learning effectiveness
- Two open-ended questions capturing qualitative feedback

The questionnaire used a five-point Likert scale where:

|| 1=Strongly Disagree || 2=Disagree || 3=Neutral || 4=Agree || 5=Strongly Agree ||

The evaluation sheet is illustrated in supporting document “Evaluation_Sheet”.

5.3 Evaluation Results

Across the ten participants, responses indicated generally positive perceptions of the system.

5.3.1 Tutorial Clarity

Participants reported that the tutorial provided a clear introduction to the interface and helped them understand how to construct proofs within the system. While most responses were positive, a small number of neutral responses suggested that additional examples could further improve clarity. The results are shown in Image G.

5.3.2 Ingredient Button Usability

Participants responded positively to the button-based formula construction system. The majority agreed that this approach simplified the creation of logical expressions and reduced the likelihood of syntax errors compared with traditional text-based input. The results are shown in Image H.

5.3.3 Workspace and Spellbook Layout

The spellbook-style workspace was generally perceived as helpful for understanding proof structure. Participants reported that the layout made it easier to follow proof progression and visualise relationships between proof lines. The results are shown in Image I.

5.3.4 Visual Feedback

Participants responded positively to visual feedback elements such as highlighting and animations. These features helped users identify relevant interface elements and provided clear indications when proof steps were accepted or rejected. The results are shown in Image J.

5.3.5 Learning Effectiveness

Several participants indicated that the interactive system improved their understanding of natural deduction concepts. Compared with traditional pen-and-paper exercises, the structured interface helped clarify how inference rules are applied within proofs. The results are shown in Image K.

5.3.6 Overall Usability and Enjoyment

Overall responses indicated that the system was easy to use and engaging to interact with. Participants particularly valued the structured formula construction and immediate validation feedback provided during proof construction. The results are shown in Image L.

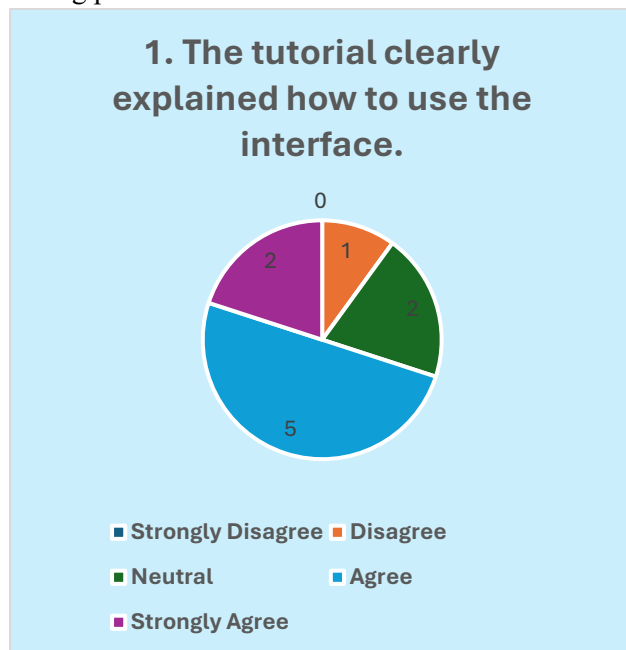


Image G

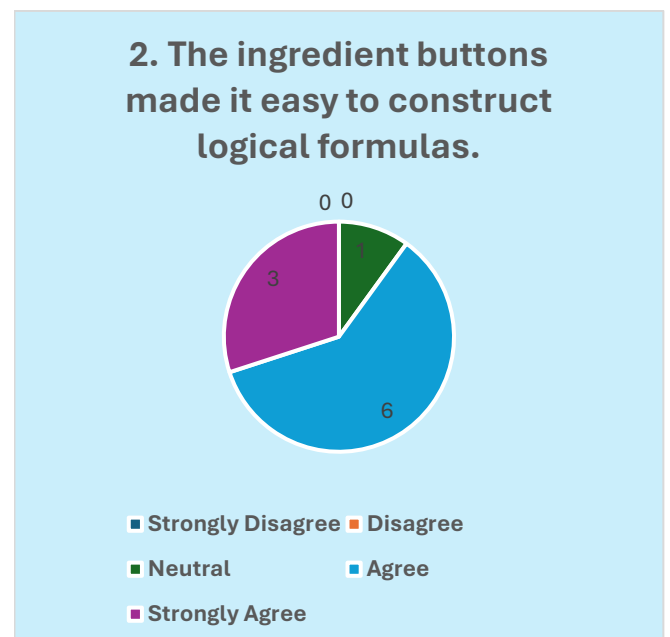


Image H



Image I

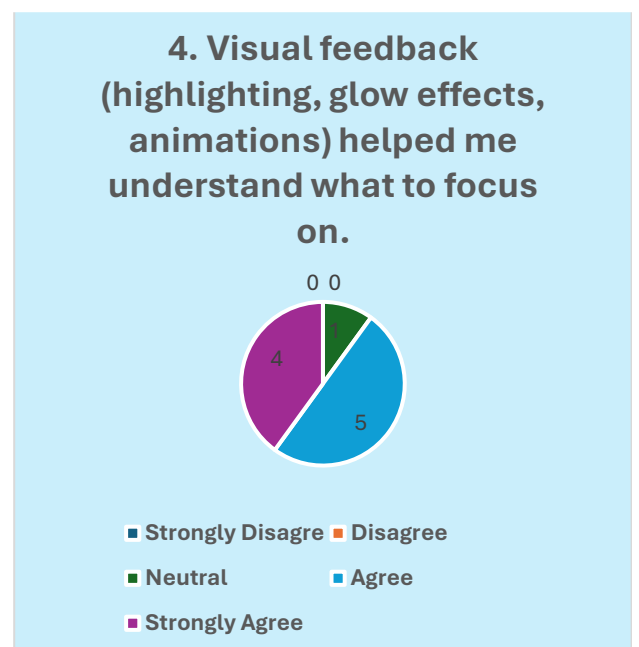


Image J

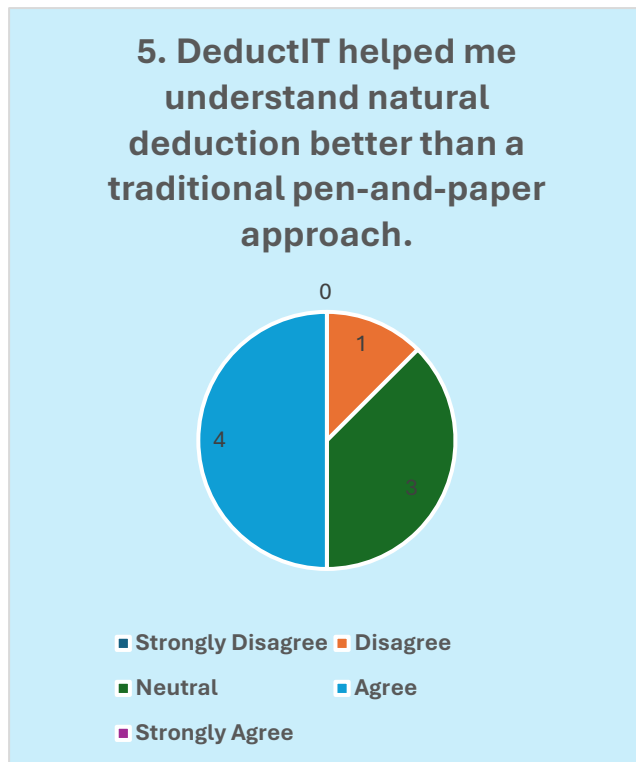


Image K

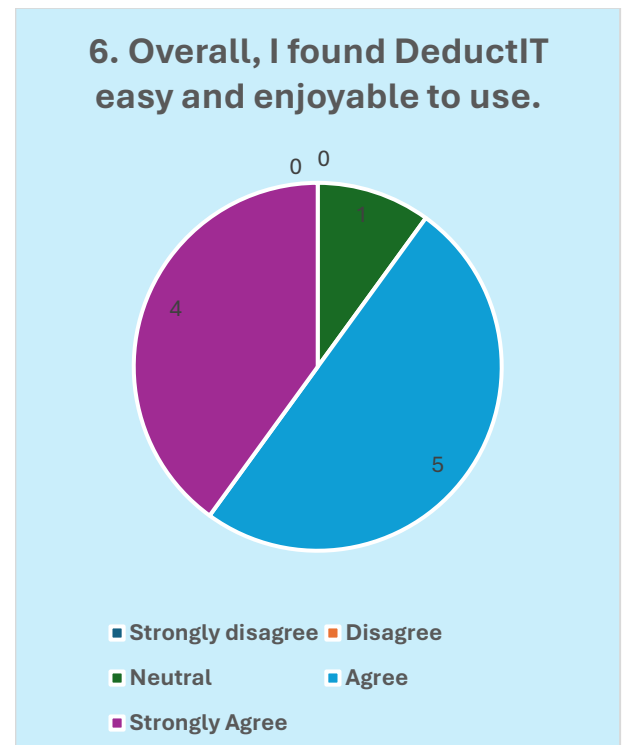


Image L

5.3.7 Summary of Quantitative Results

The numerical responses across all six Likert-scale questions indicate that participants generally had positive experiences when interacting with the system. In most categories, the majority of responses fell within the “Agree” or “Strongly Agree” range, suggesting that users found the interface intuitive and helpful when constructing natural deduction proofs.

Particularly strong responses were observed for the ingredient button system and visual feedback mechanisms, where most participants reported that these features made it easier to construct formulas and identify correct proof steps. Responses regarding the workspace layout and tutorial clarity also indicated that users were able to understand the structure of proofs and navigate the interface effectively.

Although a small number of neutral responses were recorded across several questions, very few negative responses were reported. Overall, the results suggest that the system successfully achieved its design goals of providing a clear, interactive, and engaging environment for constructing natural deduction proofs.

5.3.8 Summary of Qualitative Results

The evaluation sheet also contained 2 open ended questions at the end:

- What was the most confusing part of DeductIT, or what improvement would you suggest?
- What part of DeductIT was your favourite or think worked well?

Several participants suggested minor improvements, such as expanding the tutorial with additional examples and providing more detailed explanations when rule validation fails. Some users also recommended adding a larger variety of exercises or increasing the difficulty of later levels to further challenge experienced users.

Participants also highlighted several aspects of the system that they found particularly effective. The ingredient-based formula construction was frequently mentioned as a useful feature, as it prevented syntax errors and simplified the process of building logical expressions. The visual feedback and

structured spellbook-style workspace were also commonly identified as helpful for understanding the progression of proofs and identifying mistakes during proof construction.

A point was made regarding certain questions in the system being too difficult and out of the scope of college level. The question's which the issue was raised about were the ones with no premise' and users reported they don't believe this should be part of the question base considering it was designed for college students. This led to a revision of the question base difficulty level and many changes of questions.

Overall, the qualitative feedback suggests that users found the system both engaging and educational, while also identifying several opportunities for future refinement and expansion.

5.4 Evaluation Summary

Overall, the evaluation results indicate that the system successfully achieves its primary design goals. As shown in the evaluation results, the majority of participants reported that the system was easy to use and that the interface helped them understand the structure of natural deduction proofs. Users also responded positively to the ingredient-based formula construction and visual workspace, which helped prevent syntax errors and supported the step-by-step construction of proofs.

The qualitative feedback further reinforced these findings, with participants highlighting the clarity of the interface and the usefulness of the structured proof environment. Some suggestions for improvement were also identified, including expanding the tutorial and adjusting the difficulty of certain exercises. Overall, the evaluation suggests that the system provides an engaging and supportive environment for learning natural deduction compared with traditional static proof editors.

Chapter 6: Conclusions

6.1 Overview

This project explored the design and implementation of an interactive web-based system for learning natural deduction. Traditional proof editors often rely on text-based interfaces that can be difficult for beginners to use, particularly when constructing syntactically correct logical expressions. The goal of this project was therefore to create a system that combines formal proof validation with a more accessible and visually engaging interface.

The developed system allows users to construct proofs step-by-step using structured formula building and rule-based validation. By integrating these features into an interactive web environment, the project demonstrates how modern user interface design can improve the usability of formal logic learning tools.

6.2 Project Contributions

The main contributions of this project are:

- **An interactive natural deduction learning environment** implemented as a web application using React.
- **A structured expression builder** that prevents malformed logical formulas by enforcing syntactic constraints during construction.
- **A rule-based proof validation engine** capable of verifying propositional natural deduction steps using structural comparison of logical formulas.
- **A visually guided proof interface** incorporating animations, feedback mechanisms, and structured proof displays to improve user engagement and understanding.
- **An integrated interactive tutorial system** that guides first-time users through the interface and explains the basic mechanics of proof construction, thus reducing the learning curve..of

Together, these components form a working prototype that demonstrates how interactive visual interfaces can support the teaching of formal logic concepts.

6.4 Limitations

While the implemented system successfully demonstrates the core concept of an interactive natural deduction learning tool, several limitations should be acknowledged.

First, the current rule validation engine supports a subset of propositional natural deduction rules. Some scope-sensitive rules and more advanced logical constructs are not yet fully implemented. Second, the evaluation was conducted with a relatively small number of participants. Although the results provide useful insights into usability and user perception, a larger study would be required to draw stronger conclusions about learning effectiveness. Finally, the system currently operates without persistent user accounts or long-term data storage. As a result, user progress and performance cannot be tracked across multiple sessions. These limitations represent areas that could be addressed in future development.

6.5 Future Work

Several opportunities exist for extending the functionality and educational value of the system. One potential improvement involves introducing a backend infrastructure that allows users to create accounts and store their progress. By recording proof attempts, scores, and completion times, the system could provide long-term performance tracking and potentially support use in structured coursework or assessment environments. I believe the project could incorporate fully supporting predicate logic features as well as extending to linear temporal logic. Both of these features would drastically take the tool even further. Another promising direction involves incorporating artificial intelligence techniques for dynamic exercise generation. Currently, the system relies on a static set of predefined exercises stored in a JSON file. AI-assisted generation methods could create new proof problems automatically, allowing the system to produce different exercises each time a user interacts with it. Such an approach could also enable adaptive difficulty levels based on user performance. Future work could also focus on further enhancing the visual interface. One of the primary motivations behind this project was to make natural deduction tools more engaging than traditional proof editors. Expanding visual guidance mechanisms, such as interactive hints or more detailed proof visualisation, could further improve the learning experience. Currently the system is facing a flaw of the user being unable to go back a question, I the future I would like to add a “previous question” button so the user can move on and come back to a question if they please. Similarly, I want to add a “question complete” function which if the user was to answer a question correctly, it would mark it as complete and not show up for the user anymore if they try repeat it. Finally, extending the validation engine to fully support predicate logic would significantly increase the range of proofs that the system can handle. Implementing quantifier rules and variable scoping constraints would allow the system to support a broader range of logic exercises.

6.6 Reflection on Project Goals

At the outset of the project, the primary goal was to develop a system that makes learning natural deduction more interactive, accessible, and visually engaging than existing proof editors. The project successfully achieved several of these objectives. The implemented system provides an environment in which users can construct logical formulas using a structured interface that prevents syntactic errors. The rule validation engine enables step-by-step proof construction while ensuring logical correctness for supported rules. Another key objective was to improve the visual presentation of proof construction. Traditional natural deduction tools often present proofs as plain text structures that can be difficult for beginners to interpret. In contrast, the system developed in this project introduces visual feedback, structured proof displays, and guided formula construction to create a more engaging user experience. Feedback from the evaluation study suggests that these design choices were effective in helping users understand proof structure and interact with the system more easily. Although several planned features remain for future development, including persistent user tracking and expanded rule support, the project demonstrates that combining formal logic systems with modern

web-based interface design can produce educational tools that are both technically robust and engaging for users.

Overall, the project provides a strong foundation for further research and development in interactive systems for teaching formal reasoning and logic.

Final Statement

In conclusion, this project demonstrated how interactive digital tools can improve the accessibility and engagement of formal logic exercises. The DeductIT system provides a structured environment in which students can construct propositional natural deduction proofs step-by-step, while receiving immediate feedback through rule validation and scope checking. By combining a visual interface with progressively challenging levels, the system aims to make proof construction more intuitive and engaging for learners who may find traditional paper-based approaches difficult.

Through the design, implementation, and evaluation of DeductIT, this project illustrates how modern web technologies can be applied to support the teaching of formal logic in an interactive and educational setting. While the current system focuses on propositional logic, it establishes a strong foundation for future enhancements such as user accounts, persistent progress tracking, expanded proof systems, and the potential integration of AI-assisted exercise generation.

References

- [1] G. Gentzen, “Investigations into Logical Deduction,” 1969. [Online]. Available: https://sites.pitt.edu/~rbrandom/Courses/2022%20Phil%20of%20Language/Reasons%20texts/Gentzen%20Investigations%20Into%20Logical%20Deduction.pdf?utm_ [Accessed 12th March 2026].
- [2] M. O’Connor, “Logiko: A Game for Learning Propositional Logic,,” 2017. [Online]. Available: <https://loafey.se/logiko/>. [Accessed 5th March 2026].
- [3] O. L. Project, “Natural Deduction Proof Editor and Checker,” [Online]. Available: <https://proofs.openlogicproject.org/>. [Accessed 5th March 2026].
- [4] J. v. B. e. al, “Pandora: A Natural Deduction Proof Assistant,” [Online]. Available: <https://pandora.systems/>. [Accessed 5th March 2026].
- [5] C. M. University, “QED: Interactive Proof Checker,” [Online]. Available: https://teorth.github.io/QED/?utm_source=chatgpt.com. [Accessed 5th March 2026].
- [6] S. T. L. & J. Villadsen, “Teaching Higher-Order Logic Using Isabelle,” 8th April 2024. [Online]. Available: <https://arxiv.org/html/2404.05458v1>. [Accessed 2nd March 2026].
- [7] J. Villadsen, A. B. Jensen and A. Schlichtkrull, “NaDeA: A Natural Deduction Assistant with a Formalization in Isabelle,” 2017. [Online]. Available: <https://backend.orbit.dtu.dk/ws/portalfiles/portal/128436395/ifcolog00010.pdf>. [Accessed 2nd March 2026].
- [8] S. K. J. A. F. v. D. v. R. Leonardo de Moura, “The Lean Theorem Prover(system description),” 2015. [Online]. Available: <https://lean-lang.org/papers/system.pdf>. [Accessed 2nd March 2026].
- [9] P. Hunt, “Pete Hunt: React: Rethinking best practices-JSConf EU,” 2013. [Online]. Available: <https://2013.jsconf.eu/speakers/pete-hunt-react-rethinking-best-practices.html>. [Accessed 3rd March 2026].
- [10] F. Pfenning, “Natural Deduction,” 2023. [Online]. Available: <https://www.cs.cmu.edu/~fp/courses/15317-s23/lectures/02-natded.pdf>.

Appendices

Appendix A

Figure A.1

Proof:

Construct a proof for the argument: $P, P \rightarrow Q \therefore Q$

1		P
2		$P \rightarrow Q$
3		Q

NEW LINE

NEW SUBPROOF

Sorry there were errors.

Figure A.2

QED - an interactive textbook

Sections and exercises	
1. Conjunction introduction	1.1, 1.2
2. Conjunction elimination	2.1, 2.2(a), 2.2(b)
3. Disjunction introduction	3.1(a), 3.1(b)
4. Assumption	4.1
5. Logical connectives	5.1, 5.2
6. Deduction theorem	6.1(a), 6.1(b), 6.2
7. Push	7.1, 7.2
8. Modus ponens	8.1(a), 8.1(b), 8.2, 8.3, 8.4(a), 8.4(b), 8.4(c), 8.5(a), 8.5(b), 8.6(a), 8.6(b)
9. Case analysis	9.1(a), 9.1(b), 9.2(a), 9.2(b), 9.3(a), 9.3(b), 9.3(c), 9.4(a), 9.4(b), 9.4(c), 9.5(a), 9.5(b), 9.5(c), 9.5(d)
10. The biconditional	10.1(a), 10.1(b), 10.1(c), 10.1(d), 10.1(e), 10.1(f), 10.1(g), 10.2(a), 10.2(b), 10.2(c), 10.3, 10.4, 10.5
11. Disjunctive elimination	11.1, 11.2
12. Law of excluded middle	12.1(a), 12.1(b), 12.1(c), 12.1(d), 12.2(a), 12.2(b), 12.2(c), 12.3(a), 12.3(b), 12.4(a), 12.4(b), 12.4(c), 12.5(a), 12.5(b), 12.5(c), 12.5(d), 12.6(a), 12.6(b), 12.6(c), 12.6(d), 12.7, 12.8
13. True and false	13.1(a), 13.1(b), 13.2(a), 13.2(b), 13.3(a), 13.3(b), 13.3(c), 13.3(d)
14. Free variables	14.1
15. Push (for free variables)	15.1, 15.2(a), 15.2(b), 15.2(c)
16. Free variable introduction	16.1
17. Universal quantification	17.1, 17.2
18. Universal specification	18.1, 18.2(a), 18.2(b), 18.3(a), 18.3(b), 18.4, 18.5, 18.6
19. Existential quantification	19.1, 19.2(a), 19.2(b), 19.2(c), 19.2(d)
20. Pull	20.1, 20.2
21. Existence	21.1
22. Existential introduction	22.1(a), 22.1(b), 22.2(a), 22.2(b), 22.3(a), 22.3(b), 22.4(a), 22.4(b), 22.5(a), 22.5(b), 22.5(c), 22.5(d), 22.6(a), 22.6(b), 22.7, 22.8
23. Predicates and operators	23.1(a), 23.1(b), 23.1(c), 23.2, 23.3
24. Equality	24.1(a), 24.1(b), 24.2, 24.3, 24.4, 24.5, 24.6, 24.7, 24.8

Appendix B

Figure B.1-Use Case Diagram

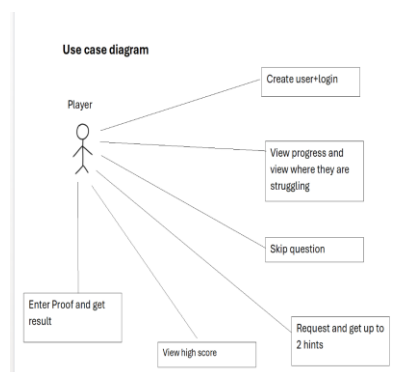


Figure B.2- Flow Diagram

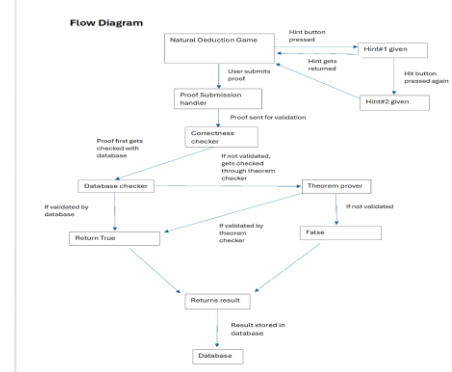


Figure B.3- Class Diagram

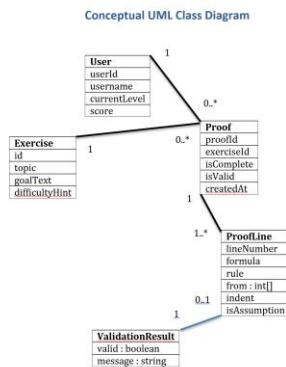
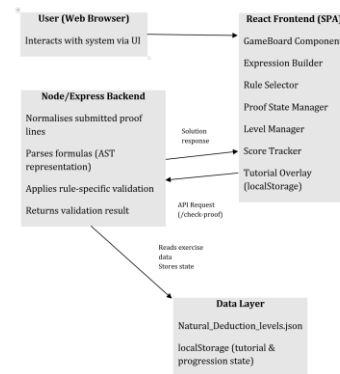


Figure B.4-Block Diagram



Appendix C

Figure C.1- Logical formulas represented as structured JavaScript objects

```

{
  type: "implication",
  left: { type: "atom", value: "P" },
  right: { type: "atom", value: "Q" }
}
  
```

Figure C.2- Example pseudocode for implication elimination

```

function validateImplicationElimination(line, proof) {
  let ref1 = proof[line.refA]
  let ref2 = proof[line.refB]

  if (ref1.formula.type === "implication" &&
    structurallyEqual(ref1.formula.left, ref2.formula) &&
    structurallyEqual(ref1.formula.right, line.formula)) {
    return true
  }

  return false
}
  
```

Appendix D

Figure D.1-Structure of JSON Level Structure

```

{
  "id": "P001",
  "topic": "Propositional Tier 1",
  "goal": "1. P & Q ⊢ P (1)",
  "solution": ""
},
{
  "id": "P002",
  "topic": "Propositional Tier 1",
  "goal": "2. P & Q ⊢ Q (1)",
  "solution": ""
},
{
  "id": "P003",
  "topic": "Propositional Tier 1",
  "goal": "3. P ⊢ P & Q (1)",
  "solution": ""
}
}
  
```

Appendix E

Figure E.1-Primary gameplay interface

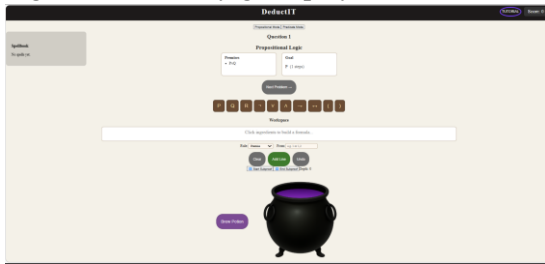


Figure E.2-Example of Nested Sub proof Structure

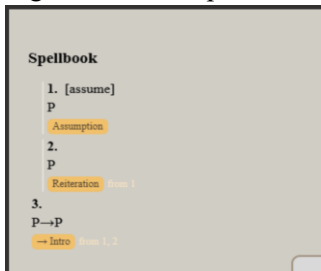


Figure E.3-Subproof Creation Buttons

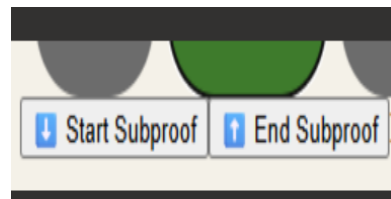


Figure E.4- rule-specific validation feedback

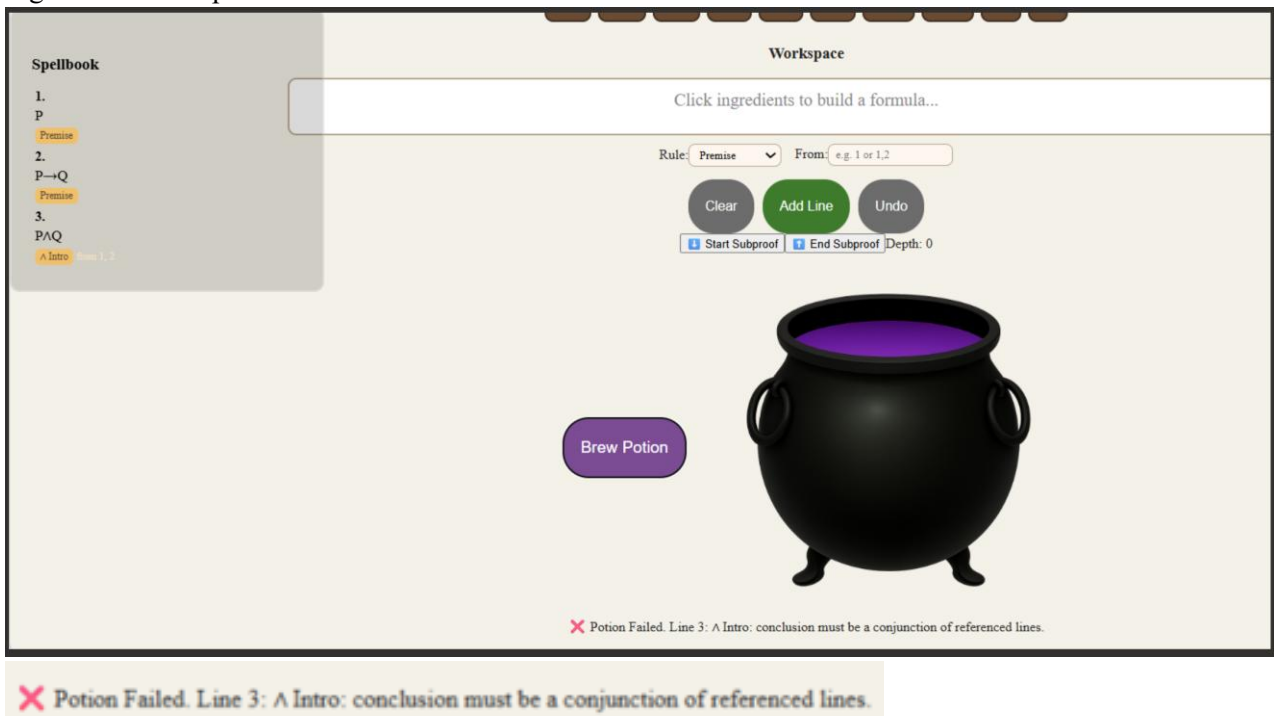
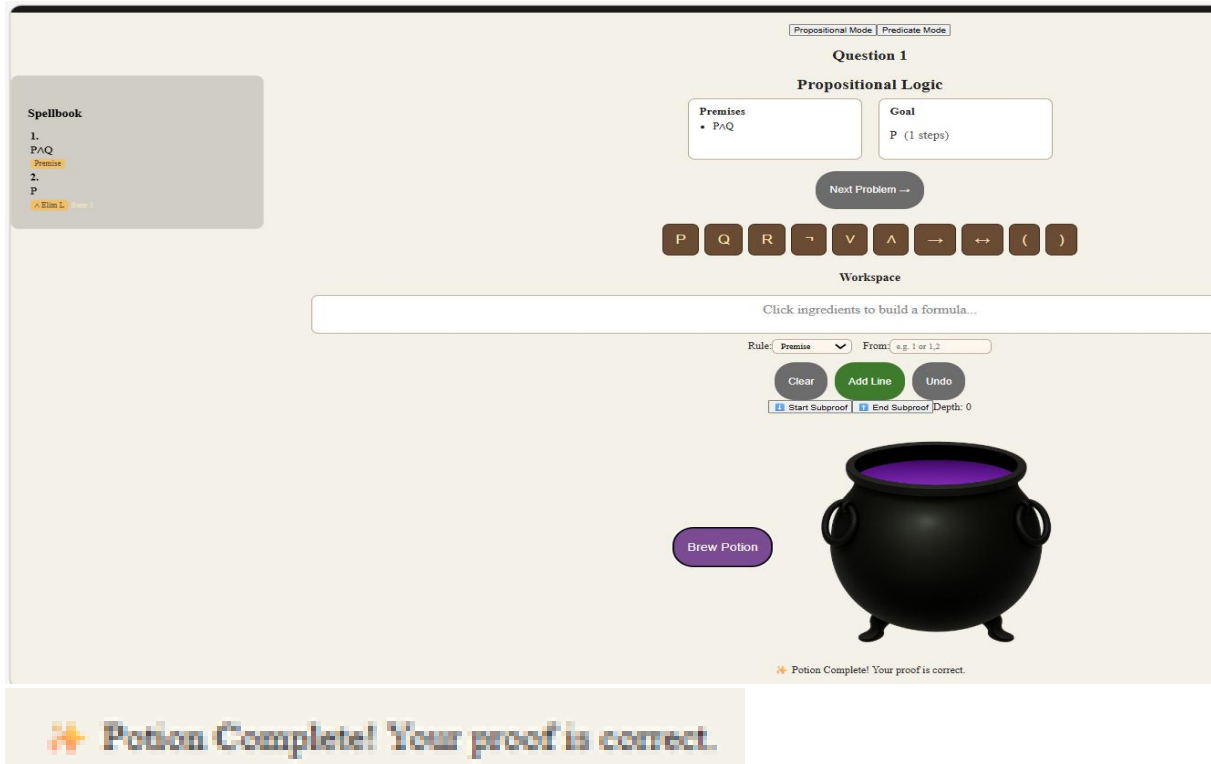
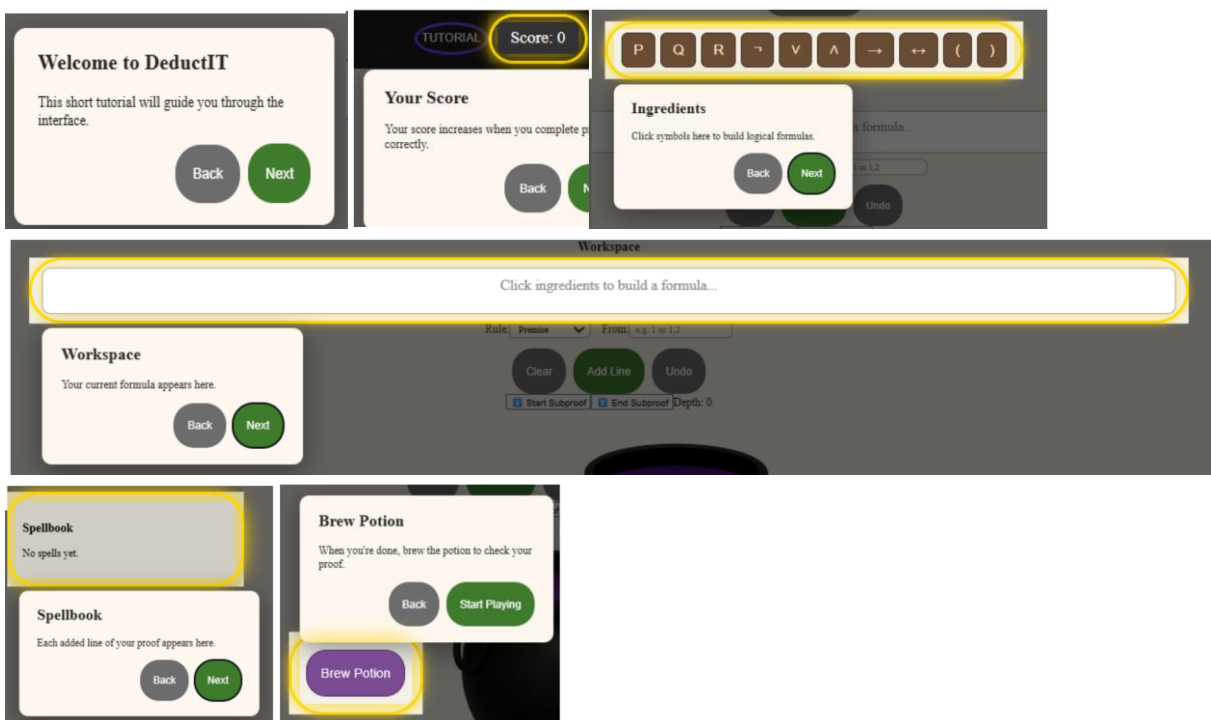


Figure E.5-Correct Proof Example



Figures E.6-Tutorial Images



Appendix F- Use of Generative AI Tools

During this project, generative AI tools were used in a limited support capacity.

Microsoft Copilot was used during development to assist with:

Code

- generating initial drafts of some JavaScript rule validation functions
- suggesting React component structures
- identifying potential syntax errors during debugging

Report

- improving clarity of written explanations
- restructuring paragraphs for readability

All generated material was reviewed, modified, and verified by the author before inclusion in the final system or report.